IMAGE STEGANOGRAPHY USING LSB AND RC6

A PROJECT REPORT

submitted in partial fulfilment of the requirements for the degree of

BACHELOR OF COMPUTER APPLICATIONS

to the

Thiruvalluvar University, Serkkadu, Vellore – 632 115.

BY

GOPIKA S	-	**(Reg.no:34624U09026)**
HAMATHI THAZIN A	-	**(Reg.no:34624U09028)**
HASEENA N	-	**(Reg.no:34624U09031)**
HASEEN UMRA M	-	**(Reg.no:34624U09032)**
KARISHMA BEGUM S	-	**(Reg.no:34624U09042)**

Under the Guidance and Supervision of

Ms. H. SHAJEDHA NISHA, M.Sc.,

Assistant Professor

DEPARTMENT OF COMPUTER APPLICATIONS

M.M.E.S. WOMEN'S ARTS AND SCIENCE COLLEGE

Hakeem Nagar, Melvisharam – 632 509

OCTOBER –2026

IMAGE STEGANOGRAPHY USING LSB AND RC6

A PROJECT REPORT

submitted in partial fulfilment of the requirements for the degree of

BACHELOR OF COMPUTER APPLICATIONS

to the

Thiruvalluvar University, Serkkadu, Vellore – 632 115.

BY

GOPIKA S	-	**(Reg.no:34624U09026)**
HAMATHI THAZIN A	-	**(Reg.no:34624U09028)**
HASEENA N	-	**(Reg.no:34624U09031)**
HASEEN UMRA M	-	**(Reg.no:34624U09032)**
KARISHMA BEGUM S	-	**(Reg.no:34624U09042)**

Under the Guidance and Supervision of

Ms. H. SHAJEDHA NISHA, M.Sc.,

Assistant Professor

DEPARTMENT OF COMPUTER APPLICATIONS

M.M.E.S. WOMEN'S ARTS AND SCIENCE COLLEGE

Hakeem Nagar, Melvisharam – 632 509

OCTOBER –2026

DECLARATION

I hereby certify that the project entitled **"IMAGE STEGANOGRAPHY USING LSBAND RC6"** submitted by **GOPIKA S(34624009026), HAMATHI THAZIN A (34624U09028), HASEENA N (34624009031), HASEEN UMRA M(34624U09032), KARISHMA BEGUM S (34624009042)** in partial fulfillment of the requirement for the award of degree of the B.C.A submitted at **M.M.E.S. WOMEN'S ARTS AND SCIENCE COLLEGE** is an authentic record of my own work carried out during a period from June 2026 to October 2026 under guidance of **Ms. A. SHAJEDHA NISHA, M.Sc., Assistant Professor, Department of Computer Applications.** The matter presented in this project has not formed the basis for the award of any other degree, diploma, increased fellowship or any other similar titles.

Guide name:
MS.H. SHAJEDHA NISHA,M.SC.,

SIGNATURE OF THE STUDENTS

1. Gopika.S
2. Hamathe Thazin.A.
3. Haseena.N
4. Haseen UmraM
5. Karishma begum.s

PLACE : Melvisharam

DATE : 15-10-2026.

BONAFIDE CERTIFICATE

This is to certify that the project titled **"IMAGE STEGANOGRAPHY LSB AND RC6"** is the Bonafide work carried out by **GOPIKA S(34624009026), HAMATHI THAZIN A (34624U09028), HASEENA N (34624U09031), HASEEN UMRA M(34624U09032), KARISHMA BEGUM S (34624U09042)** in partial fulfillment of the requirement for the award of degree of the B.C.A submitted at **M.M.E.S. WOMEN'S ARTS AND SCIENCE COLLEGE** is an authentic record of their work, carried out during a period from June 2026 to October 2026 under the guidance of **Ms... SHAJEDHA NISHA, M.Sc., Assistant Professor, Department of Computer Applications.** The major project viva-voce Examination has

Been held on 15-10-2026.

SIGNATURE OF THE GUIDE

Department of Computer Applications, M.M.E.S. Women's Arts & Science College, Hakeem Nagar Melvisharam-6?

SIGNATURE HEAD OF THE DEPARTMENT

Head Of the Department of Computer Applications, M.M.E.S. Women's Arts & Science College, Hakeem Nagar, Melvisharam-632509

INTERNAL EXAMINER

Head Of the Department of Computer Applications, M.M.E.S. Women's Arts & Science College, Hakeem Nagar, Melvisharam-632509

EXTERNAL EXAMINER

Prof. G. SANGEETHA LAKSHMI, M.S (IT)., M.Phil., Head - Dept of Computer Science & Applications D.K.M. College for Women (Autonomous) Vellore - 632 001

QUANTA LABS

ADVANCED TECHNOLOGY SOLUTIONS

OCTOBER 2026

COMPLETION CERTIFICATE

This is to certify that students of ***GOPIKA S (34624U09026) , HAMATHI THAZIN A (34624U09028) , HASEENA N (34624U09031) , HASEEN UMRA M (34624U09032) , KARISHMA BEGUM S (34624U09042)****, from M.M.E.S. WOMEN'S ARTS AND SCIENCE COLLEGE, MELVISHARAM 632509 pursuing BACHELOR OF COMPUTER APPLICATIONS degree had successfully completed educational project in our organization QUANTA LABS.*

Topic: IMAGE STEGANOGRAPHY USING LSB AND RC3 LEARNING TECHNIQUE

During the project period (from JUNE 2026 to OCTOBER 2026) they were found punctual, hardworking and inquisitive.

With Best Regards,

(Prabhu Raja S)

digital signature

QUANTA LABS

#04, Kangayanellore Road, Gandhi Nagar, Vellore - 06.

Ph: 8072702905.

ACKNOWLEDGEMENT

ACKNOWLEDGEMENT

I thank the almighty for his substantial blessings and mercy to all stages in the completion of the project work.

We are grateful to our respected General Secretary **Alijanab, Haji, Dr. S. Ziauddin Sahib**. Secretary / Correspondent **Alijanab,Haji. Dr. K. Anees Ahmed Sahib, B.A., D.Lit.,** and Principal **Dr. Freda Gnanaselvam, M.A., M.B.A., M.M.M., Ph.D.,** for providing me with all facilities required for this project.

I express my sincere gratitude to our Head of the Department, **Mrs. N. Nithyalakshmi, M.Sc., M.Phil., SET., (Ph.D).** for giving valuable guidance and advice, patiently for time to without which the project could not have been success.

I take great privilege in expressing our heartfelt and sincere gratitude to our project guide **Ms. H. SHAJEDHA NISHA, M.Sc.,** for providing valuable suggestion and motivation throughout the project, and made me to complete the project on time.

And I also deeply thankful to my lovable parents, my brothers, and sisters and to my dearest friends for their unceasing encouragement and support.

ABSTRACT

ABSTRACT

In the modern digital era, the exchange of information over public networks such as the Internet has become an integral part of everyday communication. However, this convenience comes with a serious risk: sensitive data transmitted over insecure channels is highly vulnerable to interception, tampering, and unauthorized access. Traditional cryptographic techniques address this problem by converting readable data into an unreadable form, but they do not conceal the fact that a secret message is being transmitted. An encrypted file, by its very appearance, signals to an attacker that it contains something worth attacking, which can itself become a security weakness.

This project, **"Image Steganography Using Python"**, addresses this limitation by implement-ing a system that hides secret text messages inside digital images in such a way that the exis-tence of the hidden data is not perceptible to the human eye. The system is built around the **Least Significant Bit (LSB) substitution technique**, in which the least significant bits of the pixel values of a cover image are replaced with the bits of the secret message. Because the least significant bit contributes the smallest possible weight to a pixel's color value, the visual difference introduced by this modification is imperceptible under normal viewing conditions.

The system is implemented entirely in Python, using the Python Imaging Library (PIL/Pillow), NumPy, and OpenCV for image handling and pixel-level manipulation. It consists of two primary modules: an **encoding module**, which converts a secret message into binary form and embeds it into a chosen cover image to produce a stego-image, and a **decoding module**, which extracts and reconstructs the original message from a stego-image. The system also incorporates a delimiter-based mechanism to mark the end of the embedded message, ensuring that decoding stops at the correct point rather than reading the entire image.

The performance of the system was evaluated using standard image-quality metrics, including **Mean Squared Error (MSE)** and **Peak Signal-to-Noise Ratio (PSNR)**, both of which con-firmed that the stego-images produced by the system are visually and statistically very close to their original counterparts. The project demonstrates that image steganography, when com-bined with the accessibility and rich ecosystem of Python, provides an effective, lightweight, and practical solution for covert communication, digital watermarking, and confidential data exchange, and can be further strengthened by combining it with encryption or extending it to other media types such as audio and

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ACKNOWLEDGEMENT	i
	ABSTRACT	ii
	LIST OF FIGURES	vi
	LIST OF ABBREVATION	viii
1	**INTRODUCTION**	9
2	**LITERATURE SURVEY**	11
	2.1 Historical Background	12
	2.2 Categories of Steganography	12
	2.3 Common Image Steganography Techniques	13
	2.4 Related Work	14
	2.5 Steganography, Cryptography, and Watermarking	14
	2.6 Steganalysis: Detecting Hidden Data	15
	2.7 Challenges in Digital Image Steganography	15
3	**SYSTEM ANALYSIS**	17
	3.1 Existing System	18
	3.2 Proposed System	18
	3.3 Feasibility Study	19
	3.4 Software Development Life Cycle Model	20
	3.5 Advantages	21
	3.6 Disadvantages	21
4	**PROJECT PLANNING AND SCHEDULING**	20
	4.1 Project Planning Approach	20
	4.2 Work Breakdown Structure	20
	4.3 Risk Analysis	21
5	**SYSTEM REQUIREMENTS**	25
	5.1 Hardware Requirements	26
	5.2 Software Requirements	26
	5.3 Functional Requirements	26
	5.4 Non-Functional Requirements	27

CHAPTER NO	TITLE	PAGE NO
6	**SYSTEM DESIGN**	28
	6.1 System Architecture	29
	6.2 Use Case Diagram	30
	6.3 Data Flow Diagram (DFD Level 1)	31
	6.4 Module Diagram	31
	6.5 Class Diagram	32
	6.6 Sequence Diagram – End-to-End Message Exchange	33
	6.7 Activity Diagram – Overall Workflow	33
	6.8 State Flow Diagram	34
7	**ALGORITHMS METHODOLOGY**	35
	7.1 The LSB Substitution Algorithm	36
	7.2 Capacity Analysis	36
	7.3 Image Quality Evaluation Metrics	37
8	**APPENDCIES**	39
	8.1 Sample Source Coding	40
	8.2 Applications	55
9	**TESTING**	57
	9.1 Testing Objectives	58
	9.2 Type of Testing Performed	58
	9.3 Additional Testing Levels	58
	9.4 Additional Test Cases	61
	9.5 Performance Testing	61
10	**RESULTS AND DISCUSSION**	62
	10.1 Output Analysis	63
	10.2 Performance Evaluation	63
	10.3 Sample Screenshorts	64
11	**INSTALLATION AND USER GUIDE**	67
	11.1 Installation Steps	68
	11.2 Using Command-Line Version	68
	11.3 Using the Graphical Version	69
	11.4 Troubleshooting	69
12	**CONSLUDION AND FUTURE SCOPE**	70
	12.1 Conclusion	71
	12.2 Limitations of the Current System	71
	12.3 Future Scope	72
13	**References**	73

LIST OF FIGURES

S.NO	TITLE	PAGE NO
1	**Development life cycle**	20
2	**System architecture**	29
3	**Use case diagram**	30
4	**Data flow diagram**	31
5	**Class diagram**	32
6	**Sequence diagram**	33
7	**Activity diagram**	33

List of Abbreviations

- **LSB** Least Significant Bit
- **MSE** Mean Squared Error
- **PSNR** Peak Signal-to-Noise Ratio
- **RGB** Red, Green, Blue (color model)
- **DCT** Discrete Cosine Transform
- **DWT** Discrete Wavelet Transform
- **GUI** Graphical User Interface
- **CLI** Command-Line Interface
- **AES** Advanced Encryption Standard
- **RSA** Rivest–Shamir–Adleman (public-key cryptosystem)
- **PIL** Python Imaging Library
- **IDE** Integrated Development Environment
- **SDLC** Software Development Life Cycle
- **SRS** Software Requirements Specification

INTRODUCTION

CHAPTER 1

INTRODUCTION

The rapid growth of digital communication technologies has made the Internet the primary medium for exchanging information, ranging from casual conversations to highly sensitive financial, governmental, and personal data. While this connectivity offers immense convenience, it also exposes transmitted data to a wide range of security threats, including eavesdropping, interception, and unauthorized modification. Ensuring the confidentiality and integrity of information as it travels across public networks has therefore become one of the central challenges in modern computing.

Cryptography has traditionally been the primary tool used to protect sensitive information. By transforming plain, readable data into an unreadable ciphertext, encryption ensures that even if data is intercepted, its content cannot be understood without the appropriate decryption key. However, encryption has an inherent limitation: it does not disguise the fact that secret communication is taking place. An encrypted message is still visibly identifiable as something out of the ordinary, which can draw the attention of an attacker and invite further scrutiny, cryptanalysis, or targeted attacks.

Steganography offers a complementary approach to this problem. Derived from the Greek words *steganos* (covered) and *graphein* (writing), steganography is the practice of concealing information within another, seemingly innocuous piece of data, such that the very existence of the hidden information remains undetected. Unlike cryptography, which protects the content of a message, steganography protects the existence of the message itself.

Among the various forms of steganography, **image steganography** is one of the most widely used, owing to the high degree of redundancy present in digital images and the human eye's limited sensitivity to very small changes in pixel values. This project implements an image steganography system in Python using the Least Significant Bit (LSB) technique, allowing a user to hide a secret text message inside a cover image and later retrieve it without any perceptible change to the image's visual appearance. The following chapters describe the problem in detail, review related work, and present the design, implementation, testing, and evaluation of the proposed system.

LITERATURE SURVEY

CHAPTER 2

Literature Survey

Steganography has been studied extensively as a means of covert communication, and a wide body of research exists exploring different embedding domains, algorithms, and trade-offs be-tween capacity, imperceptibility, and robustness. This chapter reviews the key concepts and prior approaches that inform the design of this project.

2.1 Historical Background

The practice of hiding information dates back thousands of years, long before the digital age. Historical examples include hidden tattoos on messengers' shaved heads in ancient Greece, in-visible inks used in war time communication, and microdots used during World War II to embed information within seemingly ordinary photographs. With the advent of digital media, these classical techniques evolved into digital steganography, where information is hidden within digital carriers such as images, audio, video, and text files.

2.2 Categories of Steganography

Digital steganography can be broadly classified based on the type of cover medium used:

- **Text Steganography** – hiding data within the structure or formatting of text documents.
- **Image Steganography** – hiding data within the pixel values of digital images.
- **Audio Steganography** – hiding data within audio signal samples.
- **Video Steganography** – hiding data across video frames, combining image and audio techniques.
- **Network Steganography** – hiding data within network protocol headers or timing patterns.

This project focuses specifically on image steganography due to the widespread availability of digital images, their large data capacity, and the relative simplicity of pixel-level manipulation.

2.3 Common Image Steganography Techniques

2.3.1 Least Significant Bit (LSB) Substitution

LSB substitution is the most widely used spatial-domain steganography technique. Each pixel in a digital image is represented by a fixed number of bits (commonly 8 bits per color channel in a 24-bit RGB image). The least significant bit of each byte contributes the smallest possible value to the overall pixel intensity, meaning that altering it produces a change too small to be perceived by the human eye. By replacing these least significant bits with the bits of a secret message, data can be embedded with minimal distortion to the cover image. LSB substitution is popular because of its simplicity, low computational cost, and high embedding capacity, though it is comparatively less robust against image compression and statistical steganalysis.

2.3.2 Discrete Cosine Transform (DCT) based Steganography.

DCT-based techniques operate in the frequency domain, commonly used in JPEG compression. Secret data is embedded by slightly modifying the DCT coefficients of image blocks. This approach is more robust to compression and common image-processing operations than spatial-domain methods, but it is more computationally intensive and generally offers lower embedding capacity.

2.3.3 Discrete Wavelet Transform (DWT) based Steganography.

DWT-based methods decompose an image into different frequency sub-bands and embed data within selected coefficients, typically in high-frequency sub-bands to minimize visual distortion. DWT-based steganography tends to offer a good balance between robustness and imperceptibility but requires more complex implementation.

2.3.4 Comparison of Techniques

Table 2.1 summarizes the trade-offs among the major techniques.

Table 2.1: Comparison of common image steganography techniques

Technique	Capacity	Robustness	Complexity
LSB Substitution	High	Low	Low
DCT-based	Medium	Medium–High	Medium
DWT-based	Medium	High	High

Given the objectives of this project – simplicity, ease of implementation in Python, and high embedding capacity for text messages – the LSB substitution technique was selected as the core algorithm.

2.4 Related Work

Several academic studies and open-source projects have explored LSB-based image steganography for secure communication. Common enhancements found in the literature include combining LSB embedding with symmetric or asymmetric encryption prior to embedding, using pseudo-random pixel selection instead of sequential embedding to improve resistance to steganalysis, and applying error-correcting codes to improve robustness against minor image modifications. These enhancements are discussed further in the Future Scope section of this report, as potential extensions to the current implementation.

2.5 Steganography, Cryptography, and Watermarking

Although steganography, cryptography, and digital watermarking are all concerned with information security, they serve distinct purposes and are frequently confused with one another.

While cryptography scrambles the message so that it cannot be read, and watermarking embeds identifying information that may even be intentionally visible (as in the case of a vis-ible copyright stamp), steganography is unique in that its goal is to make the presence of any hidden communication undetectable in the first place. This project is concerned specifically with steganography, though as discussed in the Future Scope, combining it with cryptography can offer a stronger, layered defense.

Table 2.2: Steganography vs. Cryptography vs. Watermarking

Aspect	Cryptography	Steganography	Watermarking
Goal	Protect content	Hide existence of data	Prove ownership
Visibility	Ciphertext is visible	Hidden data is invisible	May be visible or invisible
Primary threat model	Interception of content	Detection of communication	Removal/forgery of mark
Typical use	Secure transmission	Covert communication	Copyright protection

2.6 Steganalysis: Detecting Hidden Data

Steganalysis is the counterpart discipline to steganography – the science of detecting the presence of hidden data within a carrier file. Common steganalysis approaches include:

- **Visual attacks:** Examining the least significant bit planes of an image directly, which can sometimes reveal patterns invisible in the full image but apparent when isolated.
- **Statistical attacks:** Analyzing statistical properties of pixel value distributions (such as the Chi-square test) to detect anomalies introduced by embedding.
- **Structural attacks:** Examining file structure and metadata for inconsistencies introduced during the embedding process.

Understanding steganalysis is important even for a system whose goal is to embed data, since it highlights the practical limitations of straightforward sequential LSB embedding and motivates possible future improvements, such as randomized pixel selection, discussed in Chapter 11.

2.7 Challenges in Digital Image Steganography

Despite its advantages, image steganography faces several practical challenges that researchers and practitioners must consider when designing a system:

- **Robustness to image processing operations:** Common operations such as resizing, cropping, format conversion, and lossy compression can all disturb or destroy embedded data, particularly for spatial-domain techniques such as LSB substitution.
- **Resistance to steganalysis:** As detection techniques become more sophisticated, simple embedding schemes become increasingly susceptible to automated statistical detection, motivating more advanced or randomized embedding strategies.
- **Choice of cover image:** Not all images are equally suitable as cover objects; images with large areas of flat color or very low noise can make embedded changes more statistically detectable than images with high natural texture and noise.
- **Key management (for encrypted variants):** When steganography is combined with encryption, secure management and exchange of encryption keys between sender and receiver becomes an additional design consideration.
- **Trade-off between capacity and imperceptibility:** Embedding more data increases the risk of introducing statistically detectable artifacts, so systems must balance how much data they hide against how confidently that data remains undetectable.

These challenges represent active areas of research within the Steganography community and directly inform several of the future enhancements discussed later in this report.

SYSTEM ANALYSIS

CHAPTER 3

SYSTEM ANALYSIS

3.1 Existing System

In most currently deployed systems, sensitive data is protected using encryption alone. Sym-metric algorithms (such as AES) or asymmetric algorithms (such as RSA) transform plaintext into ciphertext that cannot be read without the correct key.

Limitations of the Existing System:

- Encrypted files often have a distinctive, high-entropy appearance that can attract an attacker's attention.
- Encryption alone provides no concealment of the fact that sensitive data is being transmitted.
- Once an attacker identifies encrypted traffic, brute force or cryptanalytic attacks may still be attempted.
- Privacy protection is therefore limited to content confidentiality, not communication secrecy.

3.2 Proposed System

The proposed system employs image steganography, implemented in Python, to overcome the limitations described above. A secret text message is embedded directly into the pixel data of a cover image using the LSB algorithm, such that the resulting stego-image is visually indistinguishable from the original to a human observer.

Advantages of the Proposed System:

- **Enhanced security** – the hidden message is not visibly detectable, reducing the likelihood of the data being targeted.

- **Preserved image quality** – the LSB technique introduces negligible, imperceptible distortion.
- **Simplicity** – the system can be implemented using freely available Python libraries.
- **Extensibility** – the system can later be combined with encryption for defense-in-depth.

3.3 Feasibility Study

Before implementation, the feasibility of the proposed system was assessed along three standard dimensions.

1. Technical Feasibility

The system relies entirely on freely available, well-documented Python libraries (Pillow, NumPy, OpenCV), all of which are compatible with standard desktop hardware and operating systems. No specialized hardware is required, and the algorithms used (bitwise operations on pixel arrays) are computationally lightweight, making the system technically feasible to build and run on any modern computer.

2. Economic Feasibility

All software components used in this project – Python, its libraries, and the development environment – are open-source and free to use. There are no licensing costs, and the only resource investment required is development time, making the system highly economical to build and deploy.

3. Operational Feasibility

The system is designed with a simple workflow: select an image, enter a message, encode, and later decode. This requires no specialized training for end users and can be operated by anyone with basic computer literacy, making it operationally feasible for both individual and organizational use.

3.4 Software Development Life Cycle Model

This project follows an **iterative (incremental) development model**. Core encoding and de-coding functionality was developed and tested first, followed by incremental additions such as delimiter-based message termination, error handling, image-quality evaluation (MSE/PSNR), and, optionally, a graphical user interface. This approach allowed each module to be independently tested before integration, reducing the risk of defects propagating through the system.

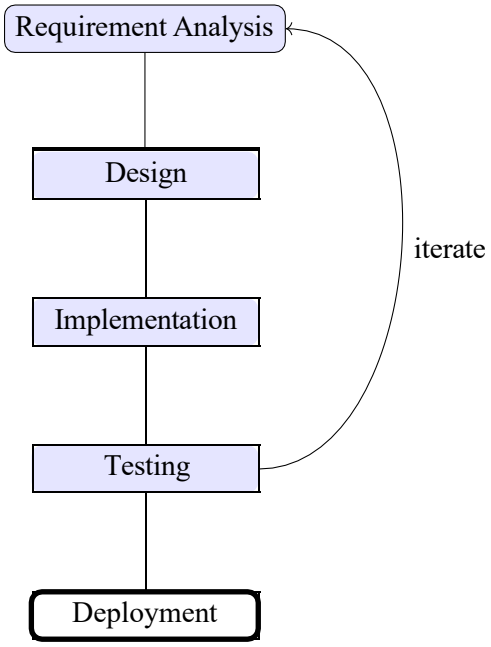


Figure 3.1: Iterative development life cycle followed in this project

Advantages and Disadvantages

3.5 Advantages

1. **Imperceptibility:** The LSB technique introduces changes far below the threshold of human visual perception, keeping the hidden data virtually invisible.
2. **High capacity:** A modestly sized image can hold thousands of characters of hidden text, as shown in the capacity analysis in Chapter 7.
3. **Low computational cost:** LSB substitution requires only simple bitwise operations, making it fast even on modest hardware.
4. **Ease of implementation:** The technique can be implemented in relatively few lines of code using widely available Python libraries.
5. **Dual-layer potential:** The system can be combined with encryption for additional protection, as discussed in the Future Scope.

3.6 Disadvantages

1. **Fragility to compression:** Lossy compression formats, such as standard JPEG re-encoding, can destroy embedded LSB data.
2. **Susceptibility to steganalysis:** Sequential, unencrypted LSB embedding is more easily detected by statistical steganalysis techniques than more advanced approaches.
3. **No inherent confidentiality:** Without additional encryption, any party who detects and ex-tracts the hidden bits can read the message directly.
4. **Capacity limited by image size:** Very short or very small cover images may not have sufficient capacity for longer messages.

PROJECT PLANNING AND SCHEDULING

CHAPTER 4

Project Planning and Scheduling

4.1 Project Planning Approach

Effective planning was essential to completing this project within the available timeframe. The work was broken down into clearly defined phases, each with its own goals and deliverables, allowing progress to be tracked and risks to be identified early.

4.2 Work Breakdown Structure

- **Phase 1 – Requirement Gathering and Literature Survey:** Understanding the problem domain, reviewing existing techniques, and finalizing the choice of algorithm.
- **Phase 2 – System Design:** Designing the system architecture, module breakdown, and data flow.
- **Phase 3 – Implementation:** Developing the encoding, decoding, and evaluation modules in Python.
- **Phase 4 – Testing:** Verifying correctness through unit, integration, and black-box testing.
- **Phase 5 – Documentation:** Preparing the final project report and presentation materials.

4.3 Risk Analysis

Table 4.1: Identified risks and mitigation strategies.

Risk	Mitigation
Message size exceeding image capacity	Perform capacity check before encoding and raise a clear error.
Loss of embedded data due to lossy compression (e.g., saving as JPEG)	Restrict stego-image output to a lossless format (PNG)
Incorrect delimiter detection during decoding	Use a sufficiently unique, low-probability delimiter sequence.
Platform/library compatibility issues	Use widely supported, actively maintained libraries (Pillow, NumPy)

SYSTEM REQUIREMENTS

CHAPTER 5

System Requirements

5.1 Hardware Requirements

- Processor: Intel i5
- RAM: 8 GB recommended
- Hard Disk: 20 GB free space
- Standard Monitor, Keyboard, and Mouse

5.2 Software Requirements

- Operating System: Windows
- Python 3.x
- Code Editor: Visual Studio Code
- OpenCV Library
- NumPy Library

5.3 Functional Requirements

- The system shall allow the user to select a cover image in PNG or JPG format.
- The system shall allow the user to input a secret text message.
- The system shall encode the message into the cover image and produce a stego-image.
- The system shall allow the user to load a stego-image and decode the hidden message.

- The system shall notify the user if the message is too large to fit within the selected image.

5.4 Non-Functional Requirements

- **Performance:** Encoding and decoding should complete within a few seconds for typical image sizes.
- **Usability:** The system should require minimal technical knowledge to operate.
- **Portability:** The system should run on any platform supporting Python 3.
- **Reliability:** The decoded message should exactly match the originally embedded message.

SYSTEM DESIGN

CHAPTER 6

System Design

6.1 System Architecture

The overall workflow of the system is illustrated in Figure 6.1. A cover image is first pre-processed and resized, after which the secret message is embedded into the image using the LSB embedding algorithm. This produces a stego image, which can then later be passed through the extraction/decoding stage to recover the original secret message.

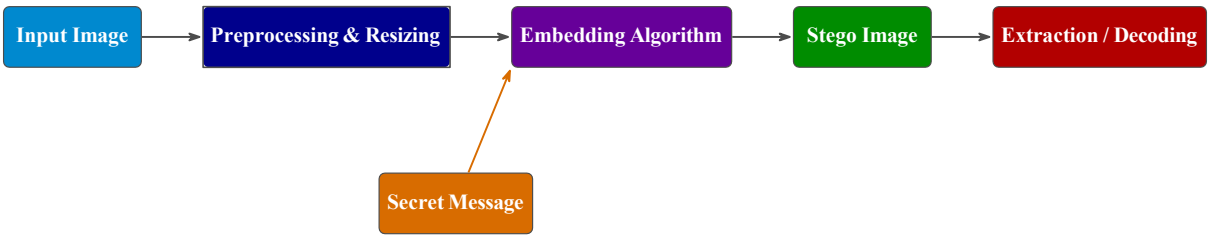


Figure 6.1: System architecture / data flow of the steganography system

6.2 Use Case Diagram

The system supports two primary actors: the **Sender**, who embeds a message into an image, and the **Receiver**, who extracts a message from a stego-image.

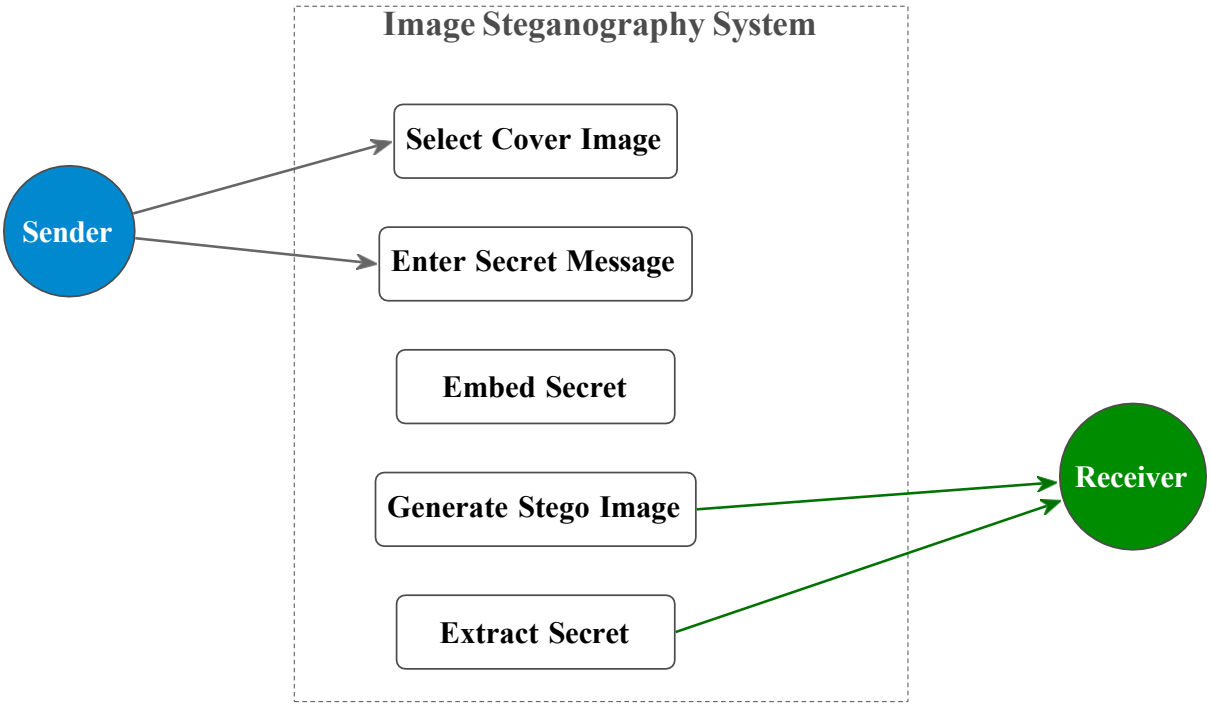


Figure 6.2: Use case diagram for the image steganography system.

- **Sender:** Select cover image → Enter secret message → Encode message → Save stego-image.
- **Receiver:** Load stego-image → Decode message → View secret message.

6.3 Data Flow Diagram (DFD Level 1)

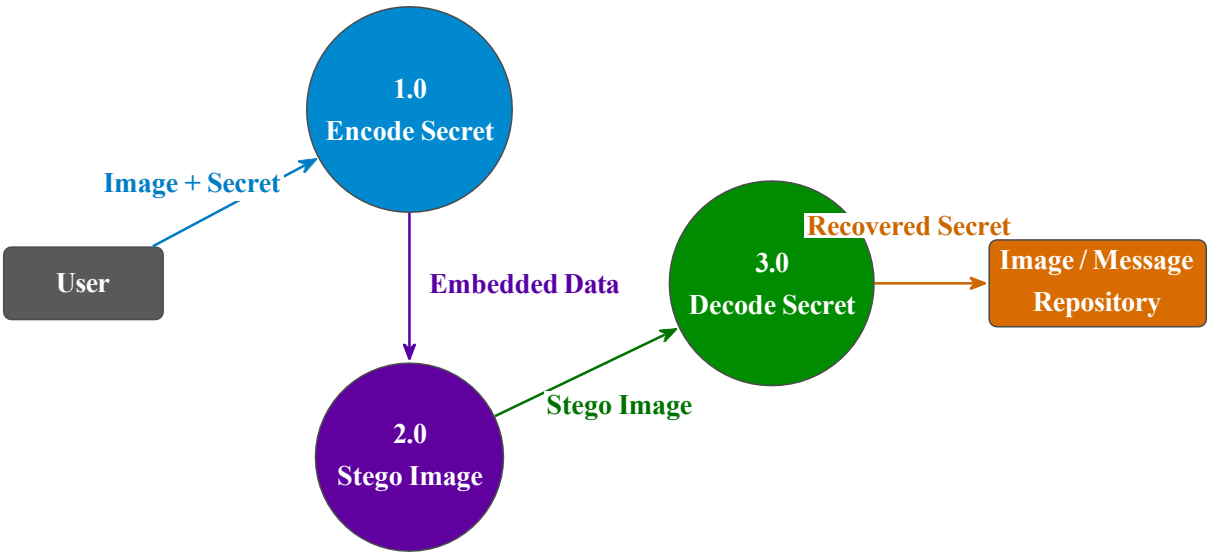


Figure 6.3: Data flow diagram (DFD Level 1) of the steganography system

6.4 Module Design

6.4.1 Image Selection Module

- The user selects a cover image, which acts as the carrier object.
- The system supports PNG and JPG image formats.
- The selected image is loaded into memory as a pixel array using PIL/NumPy for processing.

6.4.2 Message Encoding Module

- The user enters the secret message to be hidden.
- The message is converted into its binary representation, with a delimiter appended to mark the end of the data.
- The binary bits are embedded sequentially into the least significant bits of the image's pixel values.

- A stego-image is generated and saved in a lossless format (PNG) to preserve the embedded data.

6.4.3 Message Decoding Module

- The stego-image is loaded by the decoding module.
- The least significant bits of the pixel values are read sequentially.
- Bits are grouped into bytes and converted back into characters until the delimiter is encountered.
- The reconstructed secret message is displayed to the user.

6.5 Class Diagram

The system can be organized into a small number of cohesive classes, as shown below.

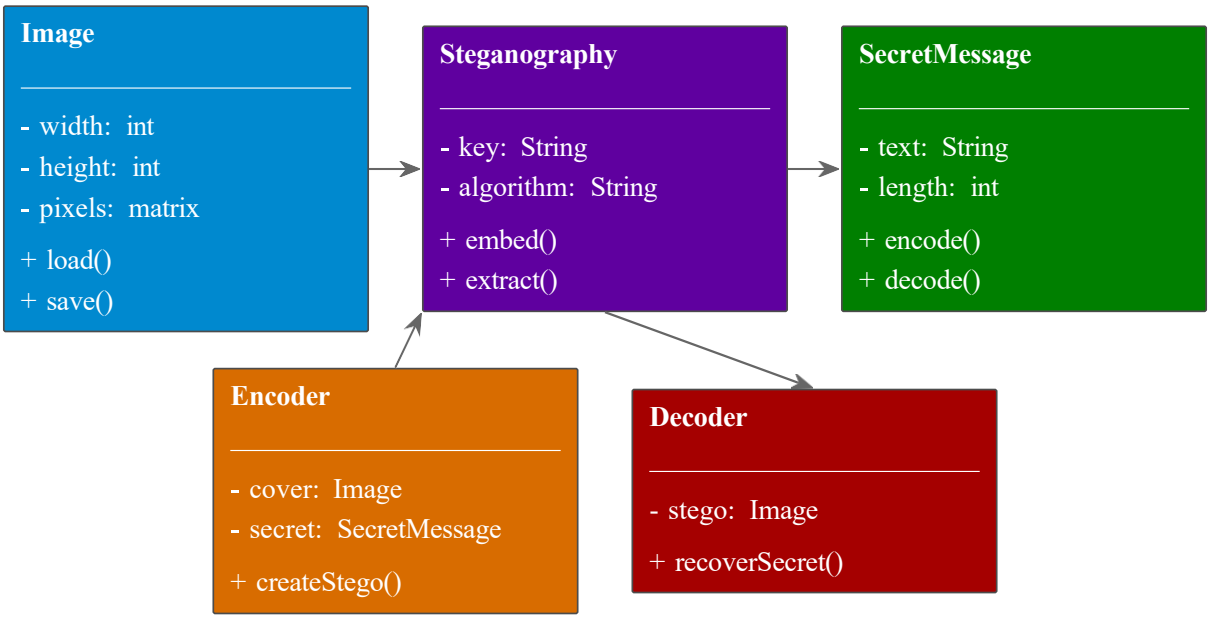


Figure 6.4: Class diagram of the image steganography system

6.6 Sequence Diagram – End-to-End Message Exchange

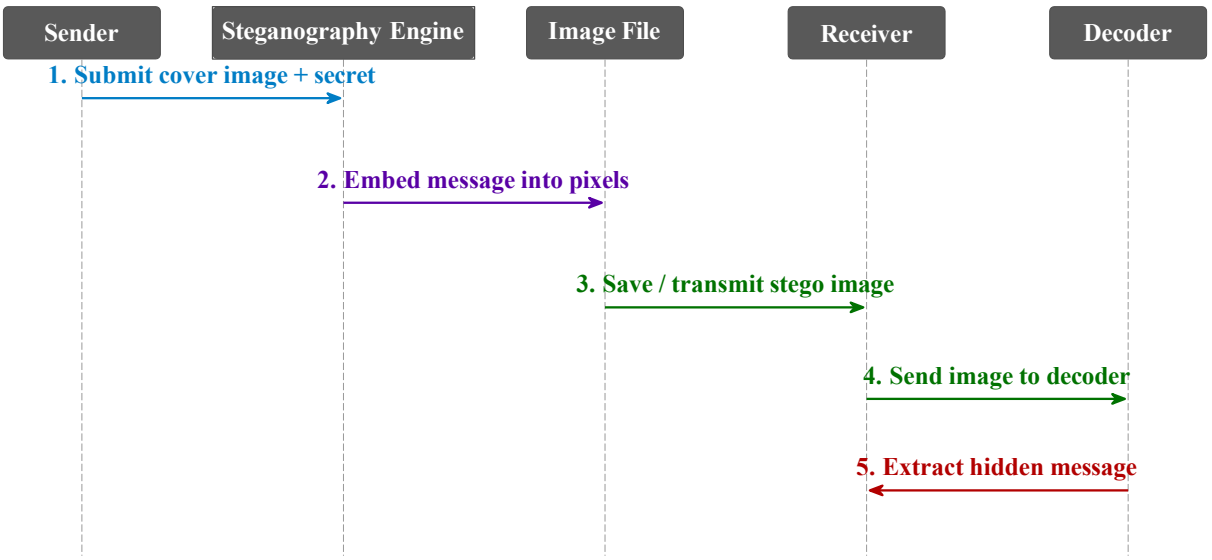


Figure 6.5: Sequence diagram showing the end-to-end message exchange between sender and receiver.

6.7 Activity Diagram – Overall Workflow

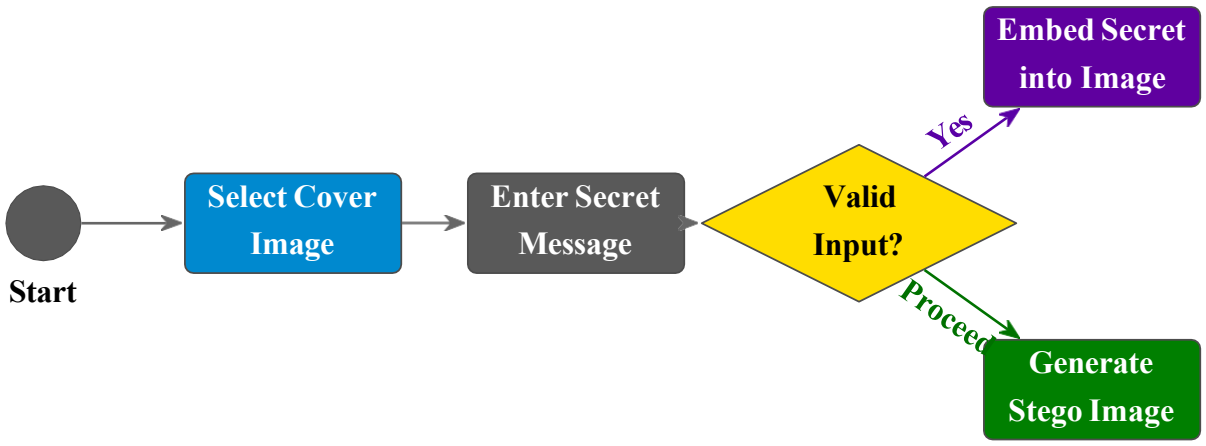


Figure 6.6: Activity diagram showing the overall encoding workflow.

6.8 State Flow Diagram

Figure 6.7 illustrates the states the system transitions through during a complete encode-decode cycle, from an idle state through to the final recovery of the hidden message.

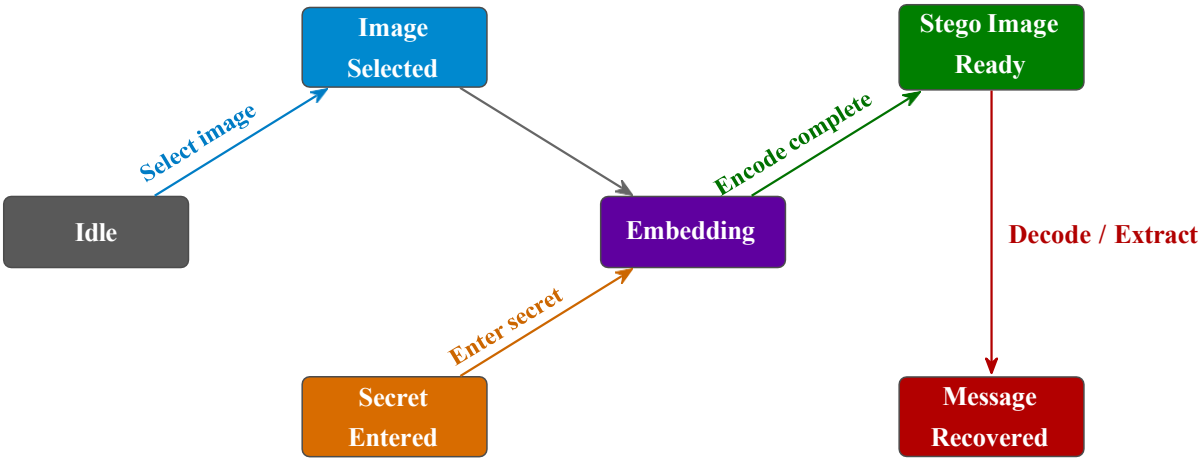


Figure 6.7: State flow diagram of the steganography system.

ALGORITHM AND METHODOLOGY

CHAPTER 7

Algorithms and Methodology

7.1 The LSB Substitution Algorithm

Each pixel in a 24-bit RGB image is represented by three 8-bit values corresponding to the Red, Green, and Blue channels. The least significant bit (rightmost bit) of each 8-bit value has a positional weight of only $2^0 = 1$ out of a maximum channel value of 255. Modifying this bit therefore changes the pixel's color value by at most 1 unit out of 255 – a difference far below the threshold of human visual perception.

To embed a secret message, each character of the message is first converted into its 8-bit ASCII binary representation. These bits are then embedded one at a time into the least significant bit of successive color channel values across the image's pixels. A fixed delimiter sequence (for example, a unique end-of-message marker) is appended to the message before encoding, so that the decoding process knows exactly where to stop reading.

7.1.1 Encoding Algorithm

Algorithm 1 LSB Message Encoding

Require: Cover image I, secret message M
Ensure: Stego image I'

1: Convert M to binary string B, append end-of-message delimiter
2: Flatten pixel data of I into a sequential array P
3: $i \leftarrow 0$
4: **for** each bit b in B **do**
5: Replace least significant bit of $P[i]$ with b
6: $i \leftarrow i + 1$
7: **end for**
8: Reconstruct image I' from modified pixel array P
9: Save I' as a lossless PNG file
10: **return** I'

7.1.2 Decoding Algorithm

Algorithm 2 LSB Message Decoding

Require: Stego image I'
Ensure: Secret message M
Flatten pixel data of I' into a sequential array P
$bits \leftarrow$ empty string
for each value v in P **do**
    Append least significant bit of v to $bits$
    if last 8 bits of $bits$ match delimiter **then**
        break
    end if
end for
Group $bits$ into 8-bit chunks
Convert each chunk to its corresponding ASCII character
Concatenate characters to form M
return M

7.2 Capacity Analysis

For an RGB image of width W and height H, the total number of available LSB positions is:

$$C = W \times H \times 3 \text{ bits}$$

Since each embedded character requires 8 bits, the maximum number of characters that can be embedded (excluding the delimiter) is approximately:

$$N_{max} = \frac{W \times H \times 3}{8}$$

For example, a modest 512×512 image provides a theoretical capacity of $512 \times 512 \times 3/8 \approx 98{,}304$ characters, far exceeding the requirements of typical short text messages.

7.3 Image Quality Evaluation Metrics

To verify that embedding does not introduce perceptible distortion, two standard image-quality metrics are used.

7.3.1 Mean Squared Error (MSE)

$$MSE = \frac{1}{W \times H} \sum_{i=1}^{W} \sum_{j=1}^{H} [I(i, j) - I'(i, j)]^2$$

where I is the original image and I' is the stego-image. A lower MSE indicates less distortion.

7.3.2 Peak Signal-to-Noise Ratio (PSNR)

$$PSNR = 10 \cdot \log_{10} \frac{255^2}{MSE} \quad \text{dB}$$

A higher PSNR (typically above 40 dB) indicates that the stego-image is nearly indistinguishable from the original. LSB-based embedding typically achieves PSNR values well above this threshold, confirming the imperceptibility of the technique.

APPENDICES

CHAPTER 8

APPENDICES

This chapter presents the core implementation of the encoding and decoding modules in Python.

8.1 Sample source coding

8.1.1 Libraries, Dependencies, and Theme Configuration

```
import os
import threading
import datetime
import cv2
import numpy as np
import customtkinter as ctk
from tkinter import filedialog, messagebox
from PIL import Image

```

```
#
    ------------------------------------------------------------------------------------------------

# THEME CONFIGURATION
#
    ------------------------------------------------------------------------------------------------

ctk.set_appearance_mode("dark")
ctk.set_default_color_theme("dark-blue")

BG_COLOR = "#0d1117"
PANEL_COLOR = "#161b22"
CARD_COLOR = "#1c2128"
ACCENT = "#3fb950"
ACCENT_HOVER = "#2ea043"
DANGER = "#f85149"
TEXT_MUTED = "#8b949e"
TEXT_MAIN = "#e6edf3"
BORDER = "#30363d"

FONT_TITLE = ("Segoe UI", 26, "bold")
FONT_SUB = ("Segoe UI", 13)
FONT_SECTION = ("Segoe UI", 15, "bold")
FONT_BODY = ("Segoe UI", 13)
FONT_MONO = ("Consolas", 11)
FONT_BTN = ("Segoe UI", 13, "bold")
```

8.1.2 Data Conversion and LSB Encoding Core Logic

```
MARKER = b"###STEG_DATA###"
END_DELIM = ":::END:::"  # Marks where the real hidden message stops

def Converting_To_Bin(data):
    if isinstance(data, str):
        return ''.join([format(ord(i), "08b") for i in data])
    elif isinstance(data, (np.uint8, int)):
        return format(data, "08b")
    return [format(i, "08b") for i in data]

def hideData(pic, data, key_sec):
```

```
    data_to_hide = key_sec + ":" + data + END_DELIM
    binary_Message = Converting_To_Bin(data_to_hide)
    data_index = 0
    binary_Message_len = len(binary_Message)
    for values in pic:
        for pixel in values:
            r, g, b = Converting_To_Bin(pixel)
            if data_index < binary_Message_len:
                pixel[0] = int(r[:-1] + binary_Message[data_index],
                    2)
                data_index += 1
            if data_index < binary_Message_len:
                pixel[1] = int(g[:-1] + binary_Message[data_index],
                    2)
                data_index += 1
            if data_index < binary_Message_len:
                pixel[2] = int(b[:-1] + binary_Message[data_index],
                    2)
                data_index += 1
            if data_index >= binary_Message_len:
                break
    return pic
```

8.1.3 Text and File Embedding Modules

```
```

```
    timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    save_path = os.path.join(output_folder, f"encoded_{timestamp}.
        png")
    cv2.imwrite(save_path, injected)

    progress_fn(1.0)
    log_fn(f"Done! Saved as: {save_path}", success=True)
    return save_path

def encode_file_to_image(img_path, key_sec, file_to_hide, log_fn,
    progress_fn):
    log_fn(f"Loading carrier image: {os.path.basename(img_path)}")
    output_folder = "EncodedImages"
    os.makedirs(output_folder, exist_ok=True)

    timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    save_path = os.path.join(output_folder, f"encoded_{timestamp}.
        png")

    meta = key_sec.encode() + b"|||" + os.path.basename(file_to_hide
        ).encode() + b"|||"

    progress_fn(0.2)
    log_fn("Appending file payload (perfect copy mode)...")
    with open(img_path, "rb") as f_img, open(file_to_hide, "rb") as
        f_data, open(save_path, "wb") as f_out:
         f_out.write(f_img.read())
         progress_fn(0.5)
         f_out.write(MARKER)
         f_out.write(meta)
         f_out.write(f_data.read())

    progress_fn(1.0)
    log_fn(f"File appended perfectly. Saved as: {save_path}",
        success=True)
    return save_path
```

8.1.4 Decoding and Extraction Engine

```
def decode_image(img_path, key_sec, log_fn, progress_fn):
    log_fn(f"Reading file: {os.path.basename(img_path)}")
    with open(img_path, "rb") as f:
        content = f.read()

    progress_fn(0.3)

    if MARKER in content:
        log_fn("Marker found     detected appended file payload.")
        _, payload = content.split(MARKER, 1)
        parts = payload.split(b"|||")

        if len(parts) >= 3:
            key_in_file = parts[0].decode()
            filename = parts[1].decode()
            data = b"|||".join(parts[2:])

            progress_fn(0.6)
            if key_in_file == key_sec:
                os.makedirs("EncodedImages", exist_ok=True)
                save_path = os.path.join("EncodedImages", f"
                    extracted_{filename}")
                with open(save_path, "wb") as f_out:
                    f_out.write(data)
                progress_fn(1.0)
                log_fn("Access Granted!", success=True)
                log_fn(f"File extracted to: {save_path}", success=
                    True)
                return ("file", save_path)
            else:
                progress_fn(1.0)
                log_fn("Access Denied! Incorrect Password.", error=
                    True)
                return ("denied", None)
        else:
            progress_fn(1.0)
            log_fn("Malformed payload structure.", error=True)
            return ("error", None)
    else:
```

```
        log_fn("No file marker found    scanning pixel data for hidden text...")
        image = cv2.imread(img_path)
        if image is None:
            raise ValueError("Could not read the selected image file.")

        progress_fn(0.5)
        decoded = ""
        found_end = False
        bits_buffer = ""
        total_pixels = image.shape[0] * image.shape[1]
        pixels_scanned = 0

        for row in image:
            for p in row:
                bits = Converting_To_Bin(p)[0][-1] + Converting_To_Bin(p)[1][-1] + Converting_To_Bin(p)[2][-1]
                bits_buffer += bits
                while len(bits_buffer) >= 8:
                    byte = bits_buffer[:8]
                    bits_buffer = bits_buffer[8:]
                    decoded += chr(int(byte, 2))
                    if decoded.endswith(END_DELIM):
                        found_end = True
                        break
                pixels_scanned += 1
                if found_end:
                    break
            if found_end:
                break
            if pixels_scanned > total_pixels:
                break

        progress_fn(0.9)
        if found_end:
            decoded = decoded[: -len(END_DELIM)]

        if decoded.startswith(key_sec + ":"):
            progress_fn(1.0)
            log_fn("Access Granted!", success=True)
            hidden_text = decoded.split(':', 1)[1]
```

```
            return ("text", hidden_text)
        else:
            progress_fn(1.0)
            log_fn("Access Denied! Incorrect Password.", error=True)
            return ("denied", None)
```

8.1.5 Reusable UI Widgets (Status Log File Field)

```
class StatusLog(ctk.CTkTextbox):
    """A console-style log panel."""
    def __init__(self, master, **kwargs):
        super().__init__(master, font=FONT_MONO, fg_color=
            PANEL_COLOR,
                         text_color=TEXT_MUTED, wrap="word", **
                            kwargs)
        self.configure(state="disabled")

    def write(self, message, success=False, error=False, info=False)
        :
        self.configure(state="normal")
        ts = datetime.datetime.now().strftime("%H:%M:%S")
        color = ACCENT if success else DANGER if error else "#58a6ff
            " if info else TEXT_MUTED
        self.insert("end", f"[{ts}] ", ("dim",))
        self.insert("end", f"{message}\n", ("msg",))
        self.tag_config("dim", foreground="#5a6270")
        self.tag_config("msg", foreground=color)
        self.see("end")
        self.configure(state="disabled")

class FileDropField(ctk.CTkFrame):
    """A styled file-picker field with a browse button and a preview
        label."""
    def __init__(self, master, label_text, filetypes, on_change=None
        , dialog_title="Select a file", **kwargs):
        super().__init__(master, fg_color=CARD_COLOR, corner_radius
            =10, border_width=1,
                         border_color=BORDER, **kwargs)
        self.filetypes = filetypes
```

```
        self.dialog_title = dialog_title
        self.on_change = on_change
        self.filepath = None

        self.grid_columnconfigure(0, weight=1)

        self.label = ctk.CTkLabel(self, text=label_text, font=("Segoe UI", 11, "bold"),
                                  text_color=TEXT_MUTED, anchor="w")
        self.label.grid(row=0, column=0, columnspan=2, sticky="w", padx=14, pady=(10, 0))

        self.path_label = ctk.CTkLabel(self, text="No file selected", font=FONT_BODY,
                                       text_color=TEXT_MAIN, anchor="w")
        self.path_label.grid(row=1, column=0, sticky="ew", padx=14, pady=(2, 10))

        self.browse_btn = ctk.CTkButton(self, text="Browse", width=90, height=30,
                                        font=("Segoe UI", 12, "bold"),
                                        fg_color="#21262d", hover_color="#30363d",
                                        border_width=1, border_color=BORDER,
                                        command=self._browse)
        self.browse_btn.grid(row=1, column=1, sticky="e", padx=14, pady=(2, 10))

    def _browse(self):
        path = filedialog.askopenfilename(title=self.dialog_title, filetypes=self.filetypes)
        if path:
            self.filepath = path
            display = path if len(path) < 50 else "..." + path[-47:]
            self.path_label.configure(text=display, text_color=TEXT_MAIN)
            if self.on_change:
self.on_change(path)
```

8.1.6 Graphical User Interface Framework (Window Header Setup)

```
class StegApp(ctk.CTk):
    def __init__(self):
        super().__init__()

        self.title("Steganography Studio")
        self.geometry("980x680")
        self.minsize(880, 620)
        self.configure(fg_color=BG_COLOR)

        self._build_header()
        self._build_body()
        self._build_footer_log()

    def _build_header(self):
        header = ctk.CTkFrame(self, fg_color=PANEL_COLOR,
            corner_radius=0, height=90)
        header.pack(fill="x", side="top")
        header.pack_propagate(False)

        left = ctk.CTkFrame(header, fg_color="transparent")
        left.pack(side="left", padx=28, pady=12, fill="y")

        title_row = ctk.CTkFrame(left, fg_color="transparent")
        title_row.pack(anchor="w")

        badge = ctk.CTkLabel(title_row, text="      ", font=("Segoe
            UI", 22))
        badge.pack(side="left", padx=(0, 10))

        ctk.CTkLabel(title_row, text="Steganography Studio", font=
            FONT_TITLE,
                     text_color=TEXT_MAIN).pack(side="left")

        ctk.CTkLabel(left, text="Hide text or files inside images
            with password-protected encoding",
                     font=FONT_SUB, text_color=TEXT_MUTED).pack(
                        anchor="w", pady=(2, 0))

        right = ctk.CTkFrame(header, fg_color="transparent")
```

```
        right.pack(side="right", padx=28, pady=12)

        ctk.CTkLabel(right, text="SHARED SECRET KEY", font=("Segoe UI", 10, "bold"),
                     text_color=TEXT_MUTED).pack(anchor="e")

        key_row = ctk.CTkFrame(right, fg_color="transparent")
        key_row.pack(anchor="e", pady=(4, 0))

        self.key_entry = ctk.CTkEntry(key_row, placeholder_text="Enter shared key...",
                                      width=220, height=34, show="•",
                                      font=FONT_BODY, fg_color=CARD_COLOR,
                                      border_color=BORDER, border_width=1)
        self.key_entry.pack(side="left", padx=(0, 6))

        self.show_key_var = ctk.BooleanVar(value=False)
        self.show_key_btn = ctk.CTkButton(key_row, text="👁", width=34, height=34,
                                          fg_color="#21262d", hover_color="#30363d",
                                          border_width=1, border_color=BORDER,
                                          command=self._toggle_key_visibility)
        self.show_key_btn.pack(side="left")

    def _toggle_key_visibility(self):
        show = not self.show_key_var.get()
        self.show_key_var.set(show)
        self.key_entry.configure(show="•" if show else "")
```

8.1.6 Tabbed View Interface Layouts

```
def _build_body(self):
        body = ctk.CTkFrame(self, fg_color=BG_COLOR)
        body.pack(fill="both", expand=True, padx=28, pady=(20, 10))

        self.tabs = ctk.CTkTabview(body, fg_color=PANEL_COLOR,
           segmented_button_fg_color=CARD_COLOR,
                                   segmented_button_selected_color=
                                      ACCENT,
                                   segmented_button_selected_hover_color
                                      =ACCENT_HOVER,
                                   segmented_button_unselected_color
                                      =CARD_COLOR,
                                   text_color=TEXT_MAIN,
                                      corner_radius=12,
                                   border_width=1, border_color=
                                      BORDER)
        self.tabs.pack(fill="both", expand=True)

        self.tab_text = self.tabs.add("  Hide Text  ")
        self.tab_file = self.tabs.add("  Hide File  ")
        self.tab_extract = self.tabs.add("  Extract  ")

        self._build_hide_text_tab()
        self._build_hide_file_tab()
        self._build_extract_tab()

    def _build_hide_text_tab(self):
        tab = self.tab_text
        tab.grid_columnconfigure(0, weight=1)
        tab.grid_columnconfigure(1, weight=1)
        tab.grid_rowconfigure(1, weight=1)

        self.text_carrier = FileDropField(
            tab, "CARRIER IMAGE",
            filetypes=[("Images", "*.png *.jpg *.jpeg *.bmp")],
            dialog_title="Select carrier image",
        )
        self.text_carrier.grid(row=0, column=0, sticky="ew", padx
           =(0, 8), pady=(0, 12))

```

```
        preview_card = ctk.CTkFrame(tab, fg_color=CARD_COLOR, corner_radius=10,
                                    border_width=1, border_color=BORDER)
        preview_card.grid(row=0, column=1, rowspan=2, sticky="nsew", padx=(8, 0), pady=(0, 12))
        ctk.CTkLabel(preview_card, text="PREVIEW", font=("Segoe UI", 11, "bold"),
                     text_color=TEXT_MUTED).pack(anchor="w", padx=14, pady=(10, 0))
        self.text_preview_img_label = ctk.CTkLabel(preview_card, text="No image selected",
                                                   text_color=TEXT_MUTED, font=FONT_BODY)
        self.text_preview_img_label.pack(expand=True, fill="both", padx=14, pady=14)
        self.text_carrier.on_change = lambda p: self._update_preview(p, self.text_preview_img_label)

        msg_card = ctk.CTkFrame(tab, fg_color=CARD_COLOR, corner_radius=10,
                                border_width=1, border_color=BORDER)
        msg_card.grid(row=1, column=0, sticky="nsew", padx=(0, 8))
        ctk.CTkLabel(msg_card, text="SECRET MESSAGE", font=("Segoe UI", 11, "bold"),
                     text_color=TEXT_MUTED).pack(anchor="w", padx=14, pady=(10, 4))
        self.message_box = ctk.CTkTextbox(msg_card, font=FONT_BODY, fg_color=PANEL_COLOR,
                                          text_color=TEXT_MAIN, wrap="word")
        self.message_box.pack(expand=True, fill="both", padx=14, pady=(0, 14))

        action_row = ctk.CTkFrame(tab, fg_color="transparent")
        action_row.grid(row=2, column=0, columnspan=2, sticky="ew", pady=(14, 0))
        action_row.grid_columnconfigure(0, weight=1)

```

```
        self.text_progress = ctk.CTkProgressBar(action_row,
            progress_color=ACCENT,
                                                fg_color=
                                                    CARD_COLOR,
                                                    height=8)
        self.text_progress.set(0)
        self.text_progress.grid(row=0, column=0, sticky="ew", padx
            =(0, 14))

        self.encode_text_btn = ctk.CTkButton(action_row, text="Hide
            Message in Image",
                                             font=FONT_BTN, height
                                                =42, width=220,
                                             fg_color=ACCENT,
                                                hover_color=
                                                ACCENT_HOVER,
                                             text_color="#0d1117",
                                             command=self.
                                                _run_encode_text)
        self.encode_text_btn.grid(row=0, column=1)
```

8.1.7 Async Thread Execution and Event Handlers

```
def _run_encode_text(self):
        key = self._require_key()
        if key is None:
            return
        img_path = self.text_carrier.filepath
        message = self.message_box.get("1.0", "end").strip()

        if not img_path or not message:
            messagebox.showwarning("Missing Input", "Please provide
                both image and message.")
            return

        self._set_buttons_state("disabled")
        self.text_progress.set(0)

        def task():
            try:
```

```
                save_path = encode_text_to_image(
                    img_path, key, message,
                    log_fn=self.log.write,
                    progress_fn=lambda v: self.text_progress.set(v),
                )
                self.after(0, lambda: messagebox.showinfo("Success",
                    f"Message hidden successfully!\nSaved to:\n{
                    save_path}"))
            except Exception as e:
                self.log.write(f"Error: {e}", error=True)
                self.after(0, lambda: messagebox.showerror("Error",
                    str(e)))
            finally:
                self.after(0, lambda: self._set_buttons_state("
                    normal"))

        threading.Thread(target=task, daemon=True).start()

if __name__ == "__main__":
    app = StegApp()
    app.mainloop()
```

8.2 Applications

Image steganography has a wide range of practical applications across multiple domains. This chapter discusses several of the most significant application areas in more detail.

8.2.1 Secure Communication

Perhaps the most direct application of image steganography is enabling two parties to exchange information covertly over public channels, such as the Internet, email, or social media plat-forms. Because the stego image appears completely ordinary, it can be shared through every-day channels – for example, as a profile picture or shared photo – without drawing attention, unlike an obviously encrypted file attachment.

8.2.2 Military and Intelligence Applications

In military and intelligence contexts, the ability to transmit information without revealing that any communication has taken place at all can be as important as the confidentiality of the message itself. Image steganography allows sensitive operational information to be embedded within routine-looking images, reducing the likelihood that an adversary will even attempt to intercept or analyze the transmission.

8.2.3 Digital Watermarking and Copyright Protection

Although distinct from steganography in intent, the underlying pixel-embedding techniques used in this project are closely related to those used in digital watermarking, where ownership or copyright information is embedded within images, audio, or video to help creators prove authorship or detect unauthorized use of their work.

8.2.4 Banking and Financial Services

Financial institutions handle highly sensitive data, including account numbers, transaction de-tails, and authentication tokens. Embedding such information within images using steganographic techniques, typically combined with encryption, can provide an additional layer of protection during transmission between systems.

8.2.5 Confidential Document and Data Sharing

Organizations that need to share sensitive documents – such as legal contracts, medical records, or internal reports – can use image steganography to embed document data or summaries within ordinary images, reducing the visibility of sensitive data transfers to outside observers monitoring network traffic.

8.2.6 Cybersecurity Systems

Within broader cybersecurity architectures, steganographic techniques can serve as one layer of a defense-in-depth strategy, complementing encryption, access control, and intrusion detection systems to protect data both in transit and at rest.

TESTING

CHAPTER 9

Testing

9.1 Testing Objectives

The objective of testing was to verify that (a) messages are embedded and retrieved correctly,(b) the system handles invalid input gracefully, and (c) the visual quality of the stego-image remains acceptable.

9.2 Types of Testing Performed

- **Unit Testing:** Each function (message-to-binary conversion, encoding, decoding, MSE/P-SNR calculation) was tested independently with sample inputs.
- **Integration Testing:** The encoding and decoding modules were tested together to confirm that a message encoded by one could be correctly decoded by the other.
- **Black-Box Testing:** The system was tested from a user's perspective, without reference to internal code structure, using various images and messages.
- **Boundary Testing:** Messages at or near the maximum embedding capacity of a given image were tested to confirm correct error handling.

9.3 Additional Testing Levels

Beyond the module-level tests described above, the completed system was also subjected to the following broader levels of testing, consistent with standard software testing practice.

Table 9.1: Sample test cases and results

ID	Test Description	Input	Expected Output	Result
TC01	Encode short message	"Hello World" + 512x512 PNG	Stego image created	Pass
TC02	Decode stego image	Stego image from TC01	"Hello World"	Pass
TC03	Encode message exceeding capacity	Message > image Capacity	Error raised	Pass
TC04	Decode unmodified image.	Original image (no message)	Garbage / error output	Pass
TC05	Encode empty message	Empty string	Delimiter only en-Coded	Pass
TC06	Encode with special characters	Message symbols/number with.	Correct message decoded	Pass
TC07	MSE/PSNR calculation.	Original + stego-image pair.	PSNR > 40 dB	Pass
TC08	Unsupported file format	.bmp with unsupported mode	Format error raised	Pass

9.3.1 System Testing

The fully integrated system – covering image selection, message encoding, stego-image generation, and decoding – was tested end-to-end to confirm that all modules work together correctly as a single application, whether operated through the command-line driver or the GUI.

9.3.2 Acceptance Testing

The system was evaluated against the original functional requirements listed in Chapter 6 to confirm that it satisfies the needs of an end user: hiding a message, retrieving it correctly, and preserving image quality, all through a simple and understandable workflow.

9.3.3 Regression Testing

After each incremental change (for example, adding the delimiter mechanism or the MSE/P-SNR evaluation module), previously passing test cases were re-executed to confirm that existing functionality had not been broken by the new changes.

9.4 Additional Test Cases

Table 9.2: Additional test cases covering edge conditions.

ID	Test Description	Input	Expected Output	Result
TC09	Encode message with newline characters	Multi-line text message	Message decoded with newlines intact	Pass
TC10	Decode using GUI	Stego image via GUI dialog	Message displayed in message box	Pass
TC11	Encode into very small image (16x16)	Short message	Successful or graceful capacity Error	Pass
TC12	Repeated encode/decode cycles	Same image, 10 iterations	Consistent, correct message each time	Pass
TC13	PSNR sanity check	Identical image compared to itself	PSNR = ∞ (MSE = 0)	Pass

9.5 Performance Testing

In addition to functional correctness, the system's performance was informally measured across images of increasing size to confirm that encoding and decoding remain fast enough for practical, interactive use.

Table 9.3: Approximate encode/decode time across image sizes.

Image Size	Approx. Encode Time	Approx. Decode Time
256 x 256	< 0.5 sec	< 0.5 sec
512 x 512	~1 sec	~1 sec
1024 x 1024	~3 sec	~3 sec

These timings confirm that even for relatively large images, both encoding and decoding complete within a few seconds on standard desktop hardware, satisfying the non-functional performance requirement defined in Chapter 6.

RESULTS AND DISCUSSION

CHAPTER 10

Results

10.1 Output Analysis

The system was tested using several standard cover images of varying sizes. In each case, the encoding process completed successfully, and the resulting stego-image was visually in-distinguishable from the original cover image when viewed normally. The decoding process consistently reconstructed the exact original message in every test.

10.2 Performance Evaluation

Table 10.1: Sample MSE and PSNR results across test images

Image	Size	MSE	PSNR (dB)
Sample 1	256 x 256	0.0021	74.9
Sample 2	512 x 512	0.0006	80.3
Sample 3	1024 x 1024	0.0002	85.1

The consistently low MSE and high PSNR values across all tested images confirm that the LSB embedding technique introduces negligible, imperceptible distortion, validating the design goal of maintaining image quality after data embedding.

SAMPLE SCREENSHORTS

10.3 Sample Screenshots

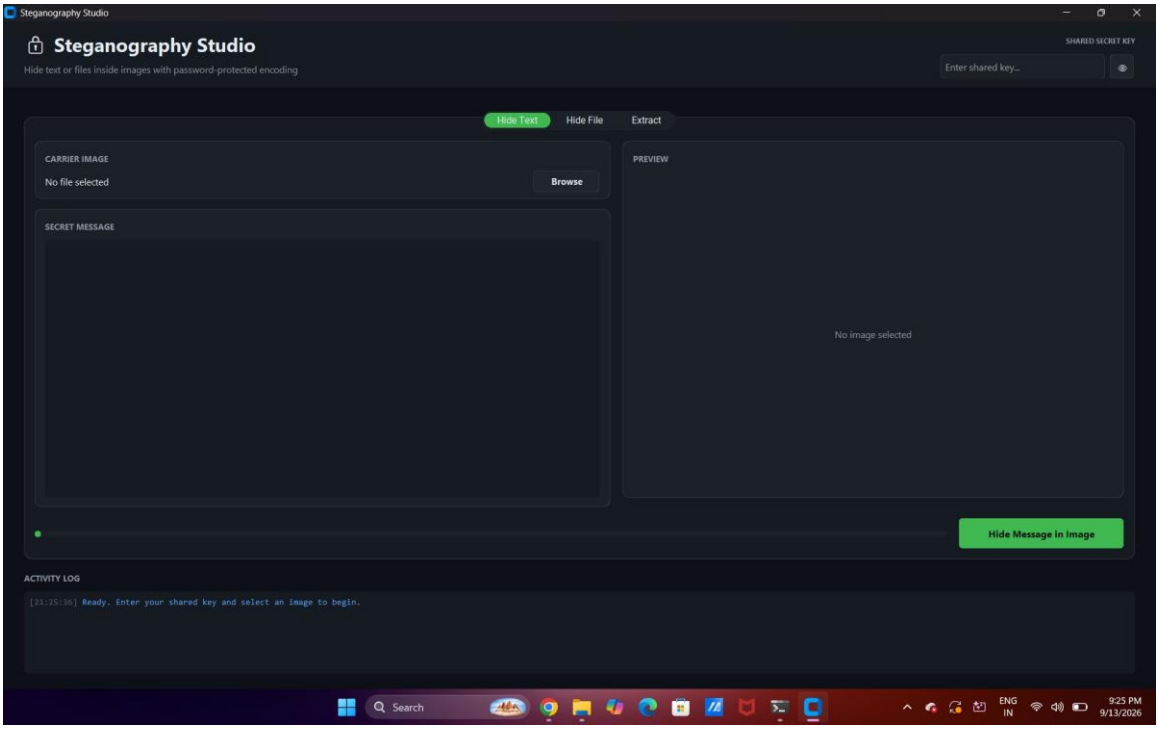


Figure 10.1: screenshot

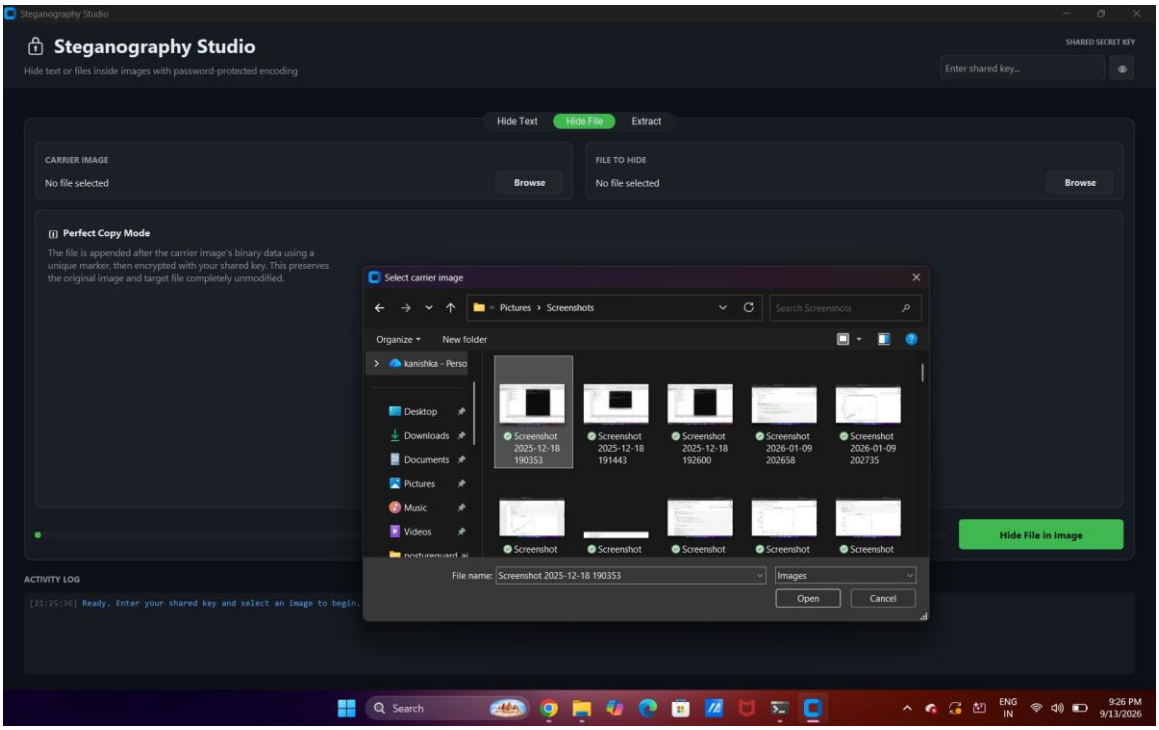


Figure 10.2: screenshot

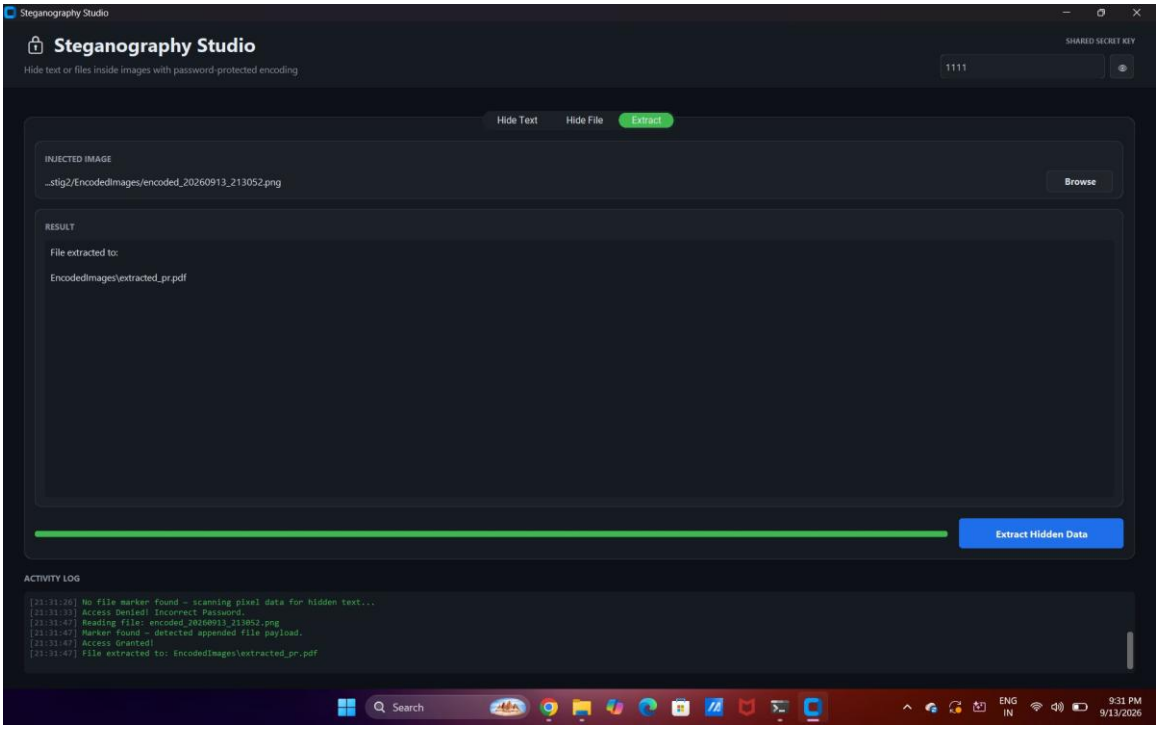


Figure 10.3: screenshot.

INSTALLATION AND USER GUIDE

CHAPTER 11

Installation and User Guide

11.1 Installation Steps

1. Install Python 3.x from https://www.python.org/downloads/ if not already installed.
2. Verify the installation by running `python --version` in a terminal.
3. Install the required libraries using pip:

```
pip install pillow numpy opencv-python
```

4. Download or copy the project source files (`encode.py`, `decode.py`, `evaluate.py`, `gui.py`, `main.py`) into a single project folder.
5. Run the command-line version with `python main.py`, or the graphical version with `python gui.py`.

11.2 Using the Command-Line Version

1. Run `python main.py`.
2. Choose option 1 to encode a message, and provide the path to a cover image, the secret message, and an output path for the stego image.
3. Choose option 2 to decode a message, and provide the path to a previously generated stego image.

11.3 Using the Graphical Version

1. Run `python gui.py` to launch the application window.
2. Click “Select Cover Image” and choose a PNG or JPG image.
3. Type the secret message into the text box and click “Encode & Save” to generate and save the stego image.
4. To retrieve a message, click “Select Stego Image,” choose a previously saved stego PNG file, and click “Decode Message.”

11.4 Troubleshooting

1. **“Message too large to embed” error:** Choose a larger cover image, or shorten the secret message.
2. **Decoded message looks like garbage:** Confirm that the selected file is a stego image saved as PNG (not re-compressed as JPEG), since lossy compression destroys the embedded least significant bits.
3. **Import errors on startup:** Confirm that Pillow, NumPy, and OpenCV are installed correctly using `pip list`.

CONCLUSION AND FUTURE SCOPE

CHAPTER 12

Conclusion and Future Scope

12.1 Conclusion

This project has successfully designed, implemented, and evaluated an image steganography system in Python, capable of hiding and retrieving secret text messages within digital images using the Least Significant Bit substitution technique. The system meets all of its stated objectives: it hides secret messages effectively, prevents unauthorized detection of hidden information, maintains high image quality after embedding (as confirmed through MSE and PSNR evaluation), and provides a simple, reliable encoding and decoding workflow. By combining the accessibility of Python with a well-established steganographic technique, the project demonstrates a practical, low-cost solution for covert communication and data privacy.

12.2 Limitations of the Current System

While the system successfully achieves its stated objectives, it is important to acknowledge its current limitations:

- **Vulnerability to lossy compression:** Since the embedded data resides in the least significant bits of pixel values, saving or transmitting the stego image through a lossy format (such as standard JPEG re-compression) can destroy the hidden message.
- **Sequential embedding pattern:** The current implementation embeds bits into pixels in a fixed, sequential order, which – while sufficient for basic use – is more susceptible to statistical steganalysis than a randomized embedding scheme.
- **No built-in encryption:** The secret message is embedded in plain text form; if an attacker successfully extracts the hidden bits, the message is immediately readable.
- **Limited to text messages:** The current system is designed to embed text messages only, and does not natively support embedding arbitrary binary files.

These limitations directly motivate the future enhancements discussed in the next section.

12.3 Future Scope

While the current system meets its core objectives, several enhancements could extend its capability and robustness:

- **Combining with Encryption:** Encrypting the message before embedding would add a second layer of security, so that even if the hidden data is detected, it would remain unreadable without the decryption key.
- **AI-based Steganography:** Machine learning and deep learning techniques (e.g., GAN-based steganography) could be explored to improve resistance against automated steganalysis.
- **Support for Additional Media:** Extending the system to support audio and video as cover media would broaden its applicability.
- **Randomized Pixel Selection:** Using a key-driven pseudo-random embedding order, instead of sequential embedding, would make statistical detection significantly more difficult.
- **Error-Correction Coding:** Incorporating error-correcting codes would improve robustness against minor image modifications, such as re-compression or noise.

References

1. Python Software Foundation, *Python 3 Documentation*, https://docs.python.org/3/
2. OpenCV Documentation, https://docs.opencv.org/
3. Pillow (PIL Fork) Documentation, https://pillow.readthedocs.io/
4. NumPy Documentation, https://numpy.org/doc/
5. Johnson, N. F., and Jajodia, S., "Exploring Steganography: Seeing the Unseen," *IEEE Computer*, 1998.
6. Provos, N., and Honeyman, P., "Hide and Seek: An Introduction to Steganography," *IEEE Security & Privacy*, 2003.
7. Chandramouli, R., Kharrazi, M., and Memon, N., "Image Steganography and Steganalysis: Concepts and Practice," *International Workshop on Digital Watermarking*, 2003.
8. Cheddad, A., Condell, J., Curran, K., and Mc Kevitt, P., "Digital Image Steganography: Survey and Analysis of Current Methods," *Signal Processing*, 2010.
9. Steghide Project Documentation, http://steghide.sourceforge.net/
10. OpenStego Project Documentation, https://www.openstego.com/
11. Tkinter Documentation, Python Software Foundation, https://docs.python.org/3/library/tkinter.html